P1 · Algoritmo genético — Panorama

Buscamos el lote diario de producción que mejor se ajusta a la demanda real de un producto.

Tipo de técnica
Metaheurística evolutiva de optimización.
Una población binaria mejora por selección, cruce y mutación.

Dataset
UCI Online Retail (CC BY 4.0)
Producto 85123A · 305 días de ventas
Demanda diaria en docenas · mediana 7



Elemento	Examen (sintético)	Esta versión (dato real)
Función objetivo	x(15-x)	1/(1+MAE(x, demanda_real))
Rango y código	0–15, 4 bits	0–31, 5 bits
Prob. mutación	0.05	0.05

Alcance: La función inversa del MAE no representa beneficio contable.

GA — Componentes y configuración

Cada individuo es un lote entero codificado en cinco bits y evaluado contra la demanda observada.

Individuo
Lote $x \in [0, 31]$ docenas
Cromosoma de 5 bits

Población inicial
4 = 00100 · 11 = 01011
14 = 01110 · 25 = 11001

Aptitud
$MAE(x) = (1/n) \Sigma |d_i - x|$
$f(x) = 1 / (1 + MAE(x))$

Operadores
Torneo por pares (4 vs 25), (11 vs 14)
Cruce tras bit 3 · mutación p = 0.05

Reemplazo y salida
2 élites + 2 hijos mutados
Salida: lote con mayor aptitud

Días evaluados (n)
305

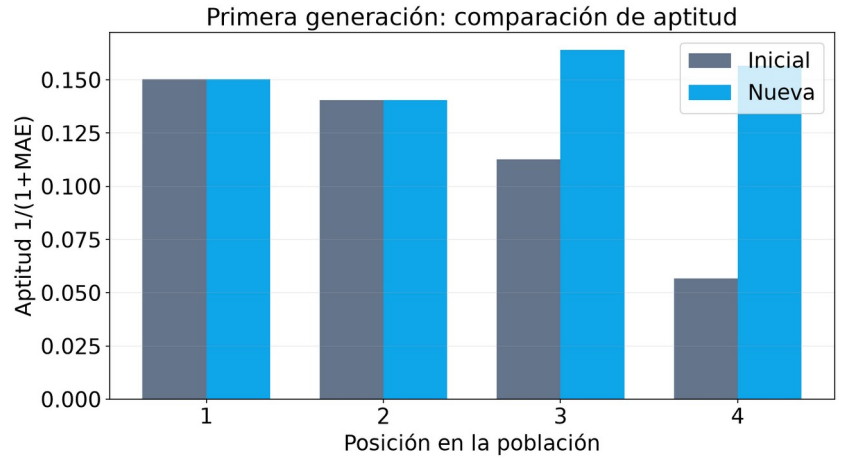
Semilla de mutación
1

Mejor lote inicial
4 docenas

GA — Primera generación paso a paso

Los cálculos de una generación responden la pregunta del examen: ¿mejoró la solución inicial?

Paso	Cálculo con los datos
1 · Evaluar x = [4, 11, 14, 25]	f = [0.150, 0.140, 0.113, 0.057]
2 · Seleccionar por torneo	(4 vs 25) → 4 · (11 vs 14) → 11
3 · Cruzar tras el bit 3	00100 × 01011 → 00111 = 7 · 01000 = 8
4 · Mutar (p = 0.05)	máscaras 00000, 00001 → hijos 7, 9
5 · Reemplazar con 2 élites	[4, 11, 7, 9] · f(7) = 1/(1 + 5.095) = 0.164



Mejor solución
4 → 7 docenas

Mejora de aptitud
+9.20%

¿Mejor que la inicial?
Sí: 0.164 > 0.150

GA — Implementación en Python

Fragmentos reales del código: cada línea marcada implementa un paso de la teoría.

`src/genetic_algorithm.py · DemandProblem y GeneticAlgorithm.run_generation`

```
  def mae(self, value: int) -> float:
      self.validate(value)
1     return float(np.mean(np.abs(self.demands - value)))
  def fitness(self, value: int) -> float:
1     return 1.0 / (1.0 + self.mae(value))
  …
2 selected = [self._winner(pair) for pair in self.tournament_pairs]
  left, right = (Individual.from_value(value).bits
                 for value in selected)
  crossed = [
3     Individual(left[: self.crossover_point]
                 + right[self.crossover_point :]).decode(),
  …
4 mask = rng.random(len(bits)) < self.mutation_probability
  …
  elites = ranked[: self.elite_count]
5 new_population = elites + mutated
```

1 Aptitud: f(x) = 1 / (1 + MAE)

2 Torneo: gana el par de mayor aptitud

3 Cruce: cabeza de un padre + cola del otro

4 Mutación: el bit cambia si u < 0.05

5 Elitismo: 2 mejores padres + 2 hijos

P2 · Lógica difusa Mamdani — Panorama

Convertimos una temperatura real de Lima en una velocidad de ventilador mediante reglas lingüísticas.

Tipo de técnica
Sistema de inferencia difusa Mamdani.
Reglas SI–ENTONCES sobre conjuntos con pertenencia parcial.

Dataset
NASA POWER · temperatura T2M diaria 2024
Lima (UNMSM) · 366 observaciones
Día evaluado 2024-05-14: 19.40 °C



Elemento	Examen (sintético)	Esta versión (dato real)
Temperatura	28 °C	19.40 °C el 2024-05-14
Universo de T	0–40 °C	16.09–22.39 °C
Salida y reglas	lento/moderado/rápido en 0–100	mismos conjuntos y tres reglas

Alcance: T2M es temperatura exterior de una celda modelada y no un sensor de aula.

Mamdani — Conjuntos y reglas

Los conjuntos de temperatura salen de los cuartiles anuales; la salida y las reglas son las del examen.

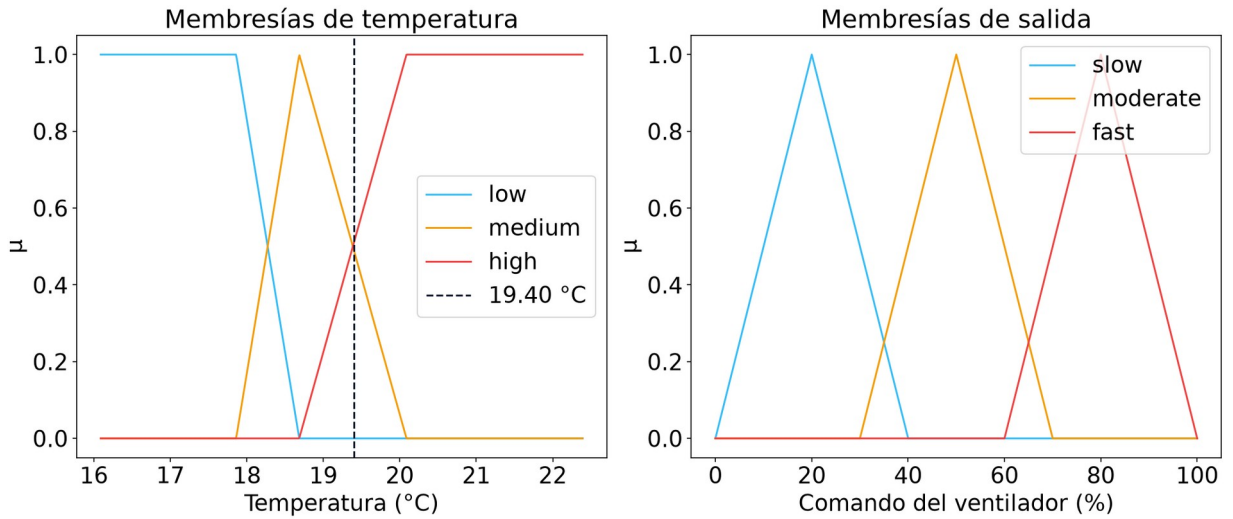
T baja · trapecio
(16.09, 16.09, 17.86, 18.68)

T media · triángulo
(17.86, 18.68, 20.09)

T alta · trapecio
(18.68, 20.09, 22.39, 22.39)

Salida V (%) · triángulos
lenta (0,20,40) · mod. (30,50,70)
rápida (60,80,100)

Reglas
baja → lenta · media → moderada
alta → rápida



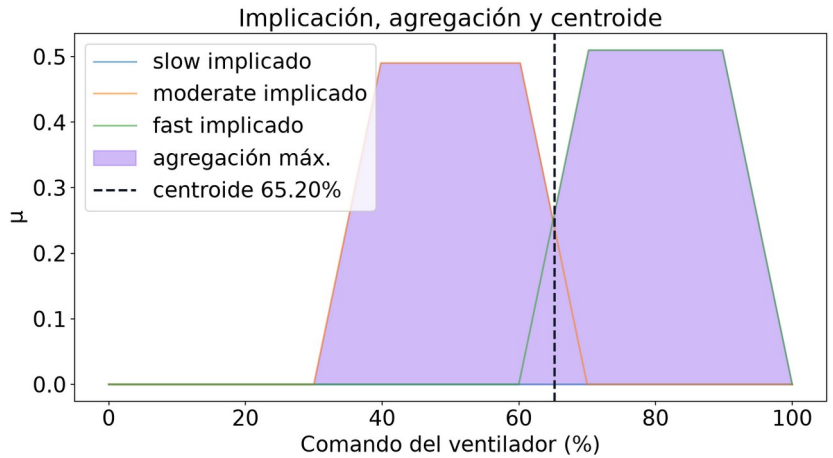
Mín · Q1 · mediana · Q3 · máx (°C)
16.1 · 17.9 · 18.7 · 20.1 · 22.4

Entrada T
19.40 °C

Mamdani — Inferencia paso a paso

Los grados de pertenencia activan reglas que se recortan, combinan y defusifican en un valor nítido.

Paso	Cálculo con los datos
1 · Fusificar T = 19.40 °C	μ_media = (20.09−19.40)/(20.09−18.68) = 0.490
2 · Evaluar reglas	α(media → moderada) = 0.490 · α(alta → rápida) = 0.510
3 · Implicar (mínimo)	μ'_B(v) = min(α, μ_B(v)) → recorta cada salida
4 · Agregar (máximo)	μ_A(v) = max(μ'_moderada(v), μ'_rápida(v))
5 · Defusificar (centroide)	v* = Σv·μ_A / Σμ_A = 187438.2 / 2874.6 = 65.20%



μ media · μ alta
0.490 · 0.510

Velocidad nítida
65.20%

Interpretación
ventilación media-alta

Mamdani — Implementación en Python

El método infer del controlador ejecuta las cuatro etapas de Mamdani sobre una malla de 0 a 100%.

`src/fuzzy_logic.py · MamdaniController.infer`

```
1 memberships = self.input_variable.memberships(input_value)
  universe = np.arange(0.0, 100.0 + self.step / 2, self.step)
  implicated: dict[str, np.ndarray] = {}
  …
  for rule in self.rules:
2     strength = memberships[rule.antecedent]
      …
      consequent = np.asarray(self.output_variable
          .sets[rule.consequent].evaluate(universe))
3     implicated[rule.consequent] = np.minimum(strength, consequent)
4 aggregated = np.maximum.reduce(list(implicated.values()))
  denominator = float(np.sum(aggregated))
  …
  numerator = float(np.sum(universe * aggregated))
5 crisp = numerator / denominator
```

1 Fusificar: μ de T en cada conjunto

2 Fuerza de la regla: α = μ(antecedente)

3 Implicación: min(α, μ_B(v)) en toda la malla

4 Agregación: máximo punto a punto

5 Centroide: Σv·μ / Σμ

P3 · Red neuronal multicapa — Panorama

Estimamos la probabilidad de que un estudiante real apruebe a partir de sus dos notas previas.

Tipo de técnica
Red feedforward 2-2-1 · aprendizaje supervisado.
Clasificación binaria con sigmoide y BCE.

Dataset
UCI Student Performance (Matemática)
395 estudiantes · split 316 / 79 estratificado
x = (G1, G2)/20 · y = 1 si G3 ≥ 10



Elemento	Examen (sintético)	Esta versión (dato real)
Entrada x_1	0.8	G1/20 = 0.75
Entrada x_2	0.6	G2/20 = 0.70
Clase esperada	aprobado	G3=15 implica y=1

Alcance: Una actualización didáctica no evalúa calidad predictiva ni generaliza a estudiantes de UNMSM.

MLP — Componentes e inicialización

La muestra, los pesos iniciales y la tasa de aprendizaje quedan declarados para reproducir el cálculo.

Muestra real

Fila 4: G1 = 15, G2 = 14

G3 = 15 → y = 1

Normalización

$x = (0.75, 0.70)$

$x_i = G_i / 20$

Capa oculta

$W^1 = [[0.1, 0.2], [-0.1, 0.1]]$

$b^1 = [0.1, -0.1]$

Capa de salida

$w^2 = [0.3, -0.2] \cdot b^2 = 0.05$

$\eta = 0.2$

Activación y pérdida

$\sigma(z) = 1 / (1 + e^{-z})$

$BCE = -[y \ln \hat{y} + (1-y) \ln(1-\hat{y})]$

Entrenamiento

316 filas

Prueba

79 filas

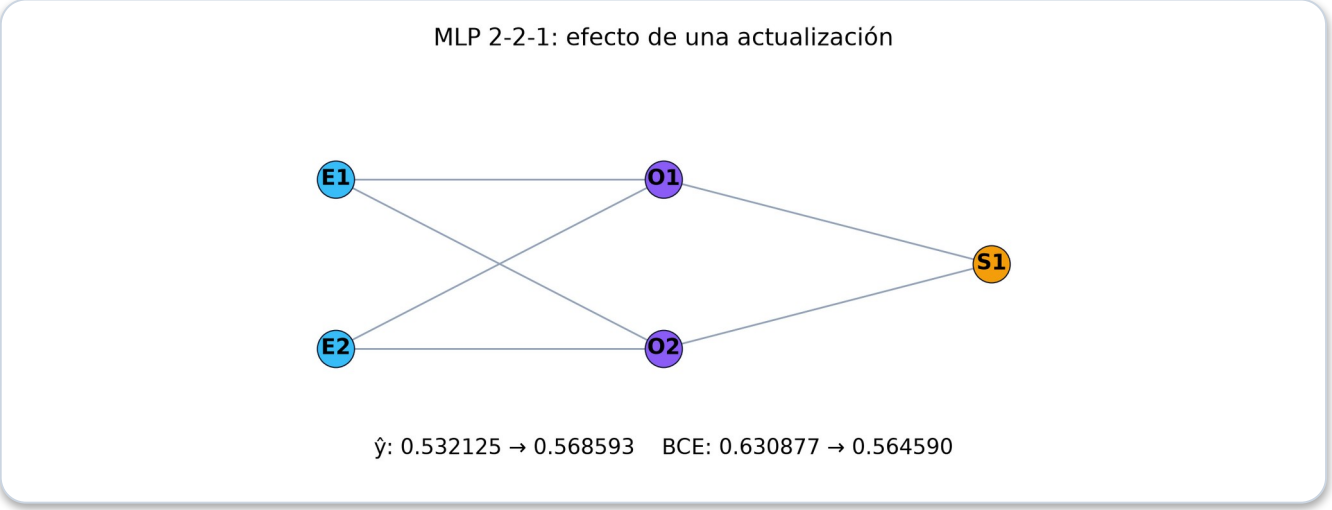
Arquitectura

2-2-1

MLP — Una iteración paso a paso

Propagación, error, retropropagación y actualización con los valores de la muestra.

Paso	Cálculo con los datos
1 · Capa oculta	$z^1 = W^1x + b^1 = [0.315, -0.105] \rightarrow a^1 = [0.578, 0.474]$
2 · Salida y pérdida	$z^2 = 0.129 \cdot \hat{y} = \sigma(z^2) = 0.532 \cdot$ BCE $= 0.631$
3 · Retropropagar	$\delta^2 = \hat{y} - y = -0.468 \cdot \delta^1 = [-0.034, 0.023]$
4 · Actualizar $\theta \leftarrow \theta - \eta\nabla\theta$	$w^2{}_1 = 0.3 - 0.2 \cdot (-0.270) = 0.354$
5 · Verificar (2.º forward)	$\hat{y}$: 0.532 → 0.569 · BCE: 0.631 → 0.565



Predicción ŷ
0.532 → 0.569

Pérdida BCE
0.631 → 0.565

MLP — Implementación en Python

Las capas y la sigmoide son objetos; train_one encadena forward, gradientes y actualización.

`src/mlp.py · MLP221.predict_with_trace y MLP221.train_one`

```
1 hidden_z = self.hidden.forward(inputs)
  hidden_a = Sigmoid.forward(hidden_z)
  output_z = float(self.output.forward(hidden_a)[0])
  prediction = float(Sigmoid.forward(np.asarray(output_z)))
  …
2 loss = BinaryCrossEntropy.value(prediction, target)
3 output_delta = prediction - target
  gradient_output_w = output_delta * hidden_a
  gradient_output_b = output_delta
4 hidden_delta = (self.output.weights[0] * output_delta) \
      * Sigmoid.derivative_from_output(hidden_a)
  gradient_hidden_w = np.outer(hidden_delta, inputs)
  …
5 self.output.weights -= learning_rate \
      * gradient_output_w.reshape(1, 2)
  …
5 self.hidden.weights -= learning_rate * gradient_hidden_w
```

1 Forward: $z = Wx + b$, $a = \sigma(z)$

2 Pérdida: entropía cruzada binaria

3 $\delta^2 = \hat{y} - y$ (sigmoide + BCE)

4 $\delta^1 = (w^2 \delta^2) \odot a^1(1 - a^1)$

5 Descenso de gradiente: $\theta \leftarrow \theta - \eta \nabla\theta$

Declaración de IA: Se utilizó asistencia de IA para planificación, programación, pruebas y redacción. La estudiante revisó los resultados y conserva la responsabilidad académica final.